



早稲田大学
WASEDA University

WASEDA University

Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫 鶴鳴

SUN Heming

Apr. 21st



Outline

- ◆ Review of last week
- ◆ How Computers Represent Negative Binary Numbers
- ◆ Control Flow

Bitwise Operators

Bitwise Operators

& bitwise AND
| bitwise inclusive OR
^ bitwise exclusive OR
<< leftshift
>> right shift
~ one's complement (unary)

$0 \wedge 0 = 0$
 $0 \wedge 1 = 1$
 $1 \wedge 0 = 1$
 $1 \wedge 1 = 0$

Bitwise AND

- ◆ The bitwise-AND operator (&) compares each bit of its first operand to the corresponding bit of its second operand. If both bits are 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

- Example

```
int nNumA=1, nNumB=3, nNumC;
```

```
nNumC = nNumA & nNumB;
```

nNumA is 00000001, nNumB is: 00000011

nNumC is now : 00000001

Bitwise Inclusive-OR

- ◆ The bitwise-inclusive-OR operator (|) compares each bit of its first operand to the corresponding bit of its second operand. If either bit is 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

- Example

```
int nNumA=9, nNumB=3, nNumC;  
nNumC = nNumA | nNumB;
```

nNumA is 00001001, nNumB is: 00000011

nNumC is now: 00001011

Bitwise Exclusive-OR

- ◆ The bitwise-exclusive-OR operator (^) compares each bit of its first operand to the corresponding bit of its second operand. If one bit is 0 and the other bit is 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

- Example

```
int nNumA=9, nNumB=3, nNumC;  
nNumC = nNumA ^ nNumB;
```

nNumA is 00001001, nNumB is: 00000011

nNumC is now: 00001010

Left shift and right shift

	Binary form	Value
<code>x=7;</code>	00000111	7
<code>x=x<<1;</code>	00001110	14
<code>x=x<<3;</code>	01110000	112
<code>x=x<<2;</code>	11000000	192

Logical Operators

Operand	Example	Meaning
&&	\$variable1 && \$variable2	Are both values true?
 	\$variable1 \$variable2	Is at least one value true?

add.c

```
#include <stdio.h>
int main(void)
{
    int x, y, a, b;
    x=1;
    y=2;
    a=x & y;
    b=x && y;
    printf("x&y is %d ¥n, x&&y is %d",a,b);
    return 0;
}
```


Exercise BMI (Body Mass Index) Program

- BMI is a controversial statistical measurement which compares a person's weight and height by :

$$\text{BMI} = \text{Weight (kg)} \div \text{Height (m)}^2$$

- BMI can be determined by using a BMI Table, .

BMI	Result
less than 18.5	Underweight
from 18.5 to 25	Normal
from 25 to 30	Overweight
over 30	Obese

Exercise BMI (Body Mass Index) Program

- BMI

$$\text{BMI} = \text{Weight}(\text{kg}) \div \text{Height}(\text{m})^2$$

- Up to now, 「height」 is inputed by cm (int) .

⇒ 「height」 should be changed from cm(int) to meter(double)



「height_m」 : height data defined by m (double)

double height_m;

the value inputed from the keyboard (「height」 (int))

should be changed to (「height_m」 (double))

✗ height_m = height / 100; → Integer division truncates any fractional part

○ height_m = height / 100.0;
 ↓ ↓
 int/ double → double

if the types of two variables are different
Low-precision -> high-precision

Exercise BMI (Body Mass Index) Program

```
...  
double height_m;  
double bmi;
```

「height」 should be changed from
cm(int) to m(double)

```
...  
height_m = height / 100.0;
```

Input height, weight
Calculate and output BMI

```
bmi = weight / (height_m * height_m);
```

```
printf("BMI : %f\n", bmi);
```

$BMI = \text{Weight (kg)} \div \text{Height (m)}^2$

```
...
```

「bmi」:double
specifier: %f

Outline

- ◆ Review of last week
- ◆ How Computers Represent Negative Binary Numbers
- ◆ Control Flow

How Computers Represent Negative Binary Numbers

Signed Magnitude

use the leftmost digit as a sign indication

00001010 = decimal 10

10001010 = decimal -10

the range of positive numbers you can store in unsigned integers is larger than signed ones.

For example,
when 32 bits is unsigned can go up to 4,294,967,296 (which is $2^{32} - 1$)

when it is signed you won't be able to use the leftmost bit. Positive range will go up to 2,147,483,647 (which is $2^{31} - 1$).

How Computers Represent Negative Binary Numbers (cont.)

The main problem with this system is that it doesn't support binary arithmetic

$$\begin{array}{r} 00001010 \text{ (decimal 10)} \\ +10001010 \text{ (decimal -10)} \\ \hline 10010100 \text{ (decimal -20)} \end{array}$$

How Computers Represent Negative Binary Numbers (cont.)

Two's complement

1. first finding the one's complement,
2. adding 1 to it.

10001100 (binary representation of -12)

11110011 (one's complement of -12)

00000001 (decimal 1)

11110100 (two's complement of -12)

How Computers Represent Negative Binary Numbers (cont.)

10001010 (binary representation of -10)

1111 0101 (one's complement of -10)
+ 00000001 (decimal 1)

1111 0110 (two's complement of -10)

00001010 (decimal 10)
+ 1111 0110 (decimal -10)

100000000 (decimal 0)

Overflow bit, throw away

Outline

- ◆ Review of last week
- ◆ How Computers Represent Negative Binary Numbers
- ◆ Control Flow

Control Flow

- ◆ Control statements enable us to specify the flow of program control; i.e., the order in which the instructions in a program must be executed. They make it possible to make decisions, to perform tasks repeatedly or to jump from one section of code to another.
 - ❖ Decision making statements
 - If-else
 - ❖ Selection Statements
 - Switch-case
 - ❖ Iteration statements
 - ❖ Jump statements

Statements and Blocks

◆ Statements

- ❖ Roughly equivalent to sentences in natural languages
- ❖ An expression such as `x = 0` or `idx++` or `printf(...)` becomes a statement when it is followed by a semicolon

- `x = 0;`
- `idx++;`
- `printf(...);`

- ## ◆ Braces { } are used to group declarations and statements together into a compound statement, or block.

- ❖ There is no semicolon after the right brace that ends a block.

Exercise

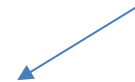
- BMI is a controversial statistical measurement which compares a person's weight and height by :

$$\text{BMI} = \text{Weight (kg)} \div \text{Height (m)}^2$$

- BMI can be determined by using a BMI Table, .

BMI	Result
less than 18.5	Underweight
from 18.5 to 25	Normal
from 25 to 30	Overweight
over 30	Obese

Determine the category and output it



If-Else

If-Else

if ~ else

```
if ( expression ) {  
    statement1;  
    statement2;  
    ...  
    statementm;  
} else {  
    statementm+1;  
    statementm+2;  
    ...  
    statementn;  
}
```



if the expression is true;
statement1, ... statementm
will be executed, otherwise,
statementm+1, ... statementn
will be executed.

•the part after 「else」 can be omitted

If-Else (cont.)

If-Else

Expression

Relational Operators

E1 less than 18.5

```
if (bmi < 18.5) {  
    ...  
}
```

E2 more than 30

```
if (bmi >= 30) {  
    ...  
}
```

Relational Operators

meaning	mathematical	operator
equal	=	==
not equal	≠	!=
less	<	<
more	>	>
under	≤	<=
upper	≥	>=

If-Else (cont.)

If-Else

Expression

from 18.5 to 25?

⇒ BMI is more than 18.5, and BMI is less than 25



Logical Operator is used for multiple expressions

Logical Operator

meaning	Operator
and	&&
or	

__ from 18.5 to 25

```
if (bmi >= 18.5 && bmi < 25) {  
    ...  
}
```

Do we need
parentheses
here?

Operators Precedence

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left

If-Else (cont.)

If-Else

```
if (bmi < 18.5) {  
    printf("Underweight¥n");  
}  
if (bmi >= 18.5 && bmi < 25) {  
    printf("Normal¥n");  
}  
if (bmi >= 25 && bmi < 30) {  
    printf("Overweight¥n");  
}  
if (bmi >= 30) {  
    printf("Obese¥n");  
}
```

BMI	Result
<18.5	Underweight
18.5–25	Normal
25–30	Overweight
>30	Obese

If-Else (cont.)

If-Else

```
if (bmi < 18.5) {  
    printf("Underweight\n");  
}  
if (bmi >= 18.5 && bmi < 25) {  
    printf("Normal\n");  
}  
    ⋮
```

Problems

- Even in the case of 「(Underweight)」 ,
the program need to be executed until the last if.
- The 1st if: 「less than 18.5」 , 2nd if: 「more than 18.5」
for the same value, more than one comparsion is needed
⇒ uselessness → reslove it by using else-if